

Pack 29 (Web + Shared) — Public Receipt Verification Portal for Third-Party Auditors (real code)

RemoteDesk · Pack 29 (Web + Shared) — Public Receipt Verification Portal for Third-Party Auditors (real code)

Codex / Claude Code handoff. Direct continuation of Packs 22 → 28. The host can now verify its own receipts (28); the server signs + protects + rotates keys (26/27). This slice opens verification to OUTSIDE parties: a public, unauthenticated page where an auditor drops in a downloaded receipt JSON and verifies its hash chain + Ed25519 signature entirely in the browser, against the org's published public key, with zero account and zero trust in our word. Build order: shared browser verify bundle → public key-fetch endpoint (unauthenticated, public material only) → public verify page → drag-drop verifier component → tests. Scope: a self-contained, client-side receipt verifier anyone can use. **No auth, no PII, public key material only. Verification runs in the auditor's browser (no server round-trip for the crypto). Content-free: the receipt itself carries no session content. Fail-closed on any mismatch.**

Tag legend: **[NEW]** = create file. **[MODIFY]** = patch existing file by hand (merge, don't overwrite).

Files created / modified / not touched

CREATED [NEW]	
packages/shared/src/permissions/receiptVerifyBundle.ts	
apps/api/src/routes/publicVerify.routes.ts	
apps/web/app/verify/page.tsx	
apps/web/app/verify/ReceiptDropzone.tsx	
apps/web/app/verify/verifyApi.ts	
apps/web/app/verify/verify.module.css	
tests/permissions/receiptVerifyBundle.test.ts	
tests/admin/publicVerify.routes.test.ts	
MODIFIED [MODIFY]	
packages/shared/src/permissions/index.ts	(export receiptVerifyBundle)
apps/api/src/server.ts	(mount public /verify routes WITHOUT auth)
NOT TOUCHED (intentionally)	
apps/desktop/**	(auditor flow is web-only)
apps/api/src/services/admin/**	(signing/keys unchanged; we only READ public keys)
packages/shared/src/permissions/{consentReceipt, receiptSignature, webcryptoEd25519}.ts	(reused as-is)

Part A — Shared: one-call browser verify bundle

FILE: packages/shared/src/permissions/receiptVerifyBundle.ts

```
import { verifyReceipt, type ConsentReceipt } from './consentReceipt';
import { verifyReceiptSignature, type SigningPublicKey } from './receiptSignature';
import { webcryptoEd25519Verify } from './webcryptoEd25519';
```

```

/**
 * One-call, browser-friendly receipt verification for third parties. Combines the
 * Pack 22 chain check + the Pack 26 signature check using WebCrypto, so an auditor
 * page can verify a downloaded receipt with a single function and no server crypto.
 * Fail-closed: any failure yields a precise, non-sensitive reason.
 */
export type VerifyBundleReason =
  | 'ok'
  | 'malformed_receipt'
  | 'chain_invalid'
  | 'unsigned'
  | 'no_public_key'
  | 'key_id_mismatch'
  | 'signature_invalid';

export interface VerifyBundleResult {
  ok: boolean;
  chainValid: boolean;
  signatureValid: boolean;
  reason: VerifyBundleReason;
  /** First broken timeline seq when the chain fails, else null. */
  firstBrokenSeq: number | null;
  /** Echoed non-sensitive facts for display. */
  facts: {
    receiptId: string | null;
    deviceId: string | null;
    rootHash: string | null;
    issuedAt: string | null;
    keyId: string | null;
  };
}

/** SHA-256 hex via WebCrypto (auditor browser). */
async function sha256Hex(input: string): Promise<string> {
  const digest = await crypto.subtle.digest('SHA-256', new TextEncoder().encode(input));
  return Array.from(new Uint8Array(digest), (b) => b.toString(16).padStart(2, '0')).join('');
}

function facts(r: Partial<ConsentReceipt> | null) {
  return {
    receiptId: r?.receiptId ?? null,
    deviceId: r?.summary?.deviceId ?? null,
    rootHash: r?.rootHash ?? null,
    issuedAt: r?.issuedAt ?? null,
    keyId: r?.signature?.keyId ?? null,
  };
}

/** Basic shape guard before any crypto. */
export function isReceiptShaped(value: unknown): value is ConsentReceipt {
  if (!value || typeof value !== 'object') return false;
  const r = value as Record<string, unknown>;
  return (
    typeof r.receiptId === 'string' &&
    typeof r.rootHash === 'string' &&
    typeof r.issuedAt === 'string' &&

```

```

    Array.isArray(r.lineItems) &&
    typeof r.summary === 'object'
  );
}

/**
 * Verify a receipt against an org id + the org's published public key. The public
 * key is fetched by the page and passed in; the crypto runs locally.
 */
export async function verifyReceiptBundle(
  orgId: string,
  receipt: unknown,
  publicKey: SigningPublicKey | null,
): Promise<VerifyBundleResult> {
  if (!isReceiptShaped(receipt)) {
    return { ok: false, chainValid: false, signatureValid: false, reason: 'malformed_receipt', firstBrokenSeq: null, facts: facts(null) };
  }
  const r = receipt as ConsentReceipt;

  const chain = await verifyReceipt(r, sha256Hex);
  if (!chain.valid) {
    return { ok: false, chainValid: false, signatureValid: false, reason: 'chain_invalid', firstBrokenSeq: chain.firstBrokenSeq, facts: facts(r) };
  }
  if (!r.signature) {
    return { ok: false, chainValid: true, signatureValid: false, reason: 'unsigned', firstBrokenSeq: null, facts: facts(r) };
  }
  if (!publicKey) {
    return { ok: false, chainValid: true, signatureValid: false, reason: 'no_public_key', firstBrokenSeq: null, facts: facts(r) };
  }
  if (publicKey.keyId !== r.signature.keyId) {
    return { ok: false, chainValid: true, signatureValid: false, reason: 'key_id_mismatch', firstBrokenSeq: null, facts: facts(r) };
  }

  const signatureValid = await verifyReceiptSignature(orgId, r, r.signature, publicKey, webcryptoEd25519Verify);
  return {
    ok: signatureValid,
    chainValid: true,
    signatureValid,
    reason: signatureValid ? 'ok' : 'signature_invalid',
    firstBrokenSeq: null,
    facts: facts(r),
  };
}

```

FILE: packages/shared/src/permissions/index.ts [MODIFY]

```

// ...existing exports through Pack 28...
export * from './receiptVerifyBundle'; // ADD

```

Part B — API: public key-fetch endpoint (unauthenticated, public material only)

FILE: apps/api/src/routes/publicVerify.routes.ts

```
import { Router } from 'express';
import { signingKeyRepository } from '../repositories/signingKeyRepository';

const h = (fn: (req: any, res: any) => Promise<void>) => (req: any, res: any, next: any) => fn(req, res).catch(next);

/**
 * PUBLIC verification routes. NO AUTH — but they expose ONLY public key material
 * by keyId. This lets a third-party auditor fetch the exact public key a receipt
 * was signed with and verify entirely in their browser.
 *
 * SECURITY: only publicKeyById is used here. It selects public columns only;
 * there is NO path that returns private/encrypted key material.
 */
export const publicVerifyRoutes = Router()
  /** Fetch a published public key by its keyId (rotation-safe: old keys resolvable). */
  .get('/signing-key/:keyId', h(async (req, res) => {
    const pub = await signingKeyRepository.publicKeyById(req.params.keyId);
    if (!pub) {
      res.status(404).json({ error: 'unknown_key' });
      return;
    }
    // Public material only — keyId, publicKey, alg, timestamps.
    res.json(pub);
  })))
  /** Lightweight liveness for the verify page. */
  .get('/health', h(async (_req, res) => {
    res.json({ ok: true });
  })));
```

FILE: apps/api/src/server.ts [MODIFY]

```
// ...existing imports...
import { publicVerifyRoutes } from '../routes/publicVerify.routes';

// ---- ADD: mount PUBLIC verify routes WITHOUT requireAuth/orgContext ----
// Place this BEFORE any global auth middleware, or ensure these paths are
// allow-listed by your auth layer. Public, read-only, public-key-material only.
// app.use('/verify', publicVerifyRoutes);
//
// SECURITY NOTE: do NOT mount any admin/private route under /verify. The only
// key endpoint here is publicKeyById (public columns).
```

Part C — Web: public verify page

FILE: apps/web/app/verify/verifyApi.ts

```
import type { SigningPublicKey } from '@remotedesk/shared';

/**
```

```

* Public verify page API. Fetches ONLY the published public key by keyId. Uses a
* configurable base URL so the verify page can be hosted standalone if desired.
*/
const BASE = process.env.NEXT_PUBLIC_VERIFY_API_BASE ?? '';

export const verifyApi = {
  async publicKey(keyId: string): Promise<SigningPublicKey | null> {
    try {
      const res = await fetch(`${BASE}/verify/signing-key/${encodeURIComponent(keyId)}`);
      if (!res.ok) return null;
      return (await res.json()) as SigningPublicKey;
    } catch {
      return null;
    }
  },
};

```

FILE: apps/web/app/verify/ReceiptDropzone.tsx

```

'use client';

import React, { useCallback, useState } from 'react';
import styles from './verify.module.css';

/**
 * Drag-and-drop (or pick) a receipt JSON file. Parses locally and hands the
 * parsed object up. No upload — the file never leaves the browser.
 */
export function ReceiptDropzone({ onReceipt }: { onReceipt: (parsed: unknown, rawName: string) => void }) {
  const [dragging, setDragging] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const handleFile = useCallback(async (file: File) => {
    setError(null);
    try {
      const text = await file.text();
      onReceipt(JSON.parse(text), file.name);
    } catch {
      setError('That file is not valid JSON.');
```

```

    <p className={styles.privacy}>The file is verified in your browser. Nothing is uploaded.</p>
    {error && <div className={styles.error}><error></div>}
  </div>
);
}

```

FILE: apps/web/app/verify/page.tsx

```

'use client';

import React, { useState } from 'react';
import { verifyReceiptBundle, type VerifyBundleResult, type ConsentReceipt } from '@remotedesk/shared';
import { ReceiptDropzone } from './ReceiptDropzone';
import { verifyApi } from './verifyApi';
import styles from './verify.module.css';

const REASON_TEXT: Record<string, string> = {
  ok: 'Receipt is authentic: hash chain intact and signature valid.',
  malformed_receipt: 'This file is not a RemoteDesk consent receipt.',
  chain_invalid: 'The receipt timeline has been altered (hash chain broken).',
  unsigned: 'The receipt has no signature (chain is intact, but not signed).',
  no_public_key: 'Could not fetch the signing public key for this receipt.',
  key_id_mismatch: 'The fetched key does not match the receipt signature key id.',
  signature_invalid: 'The signature does not match (possible tampering or wrong org).',
};

/**
 * Public, no-account receipt verification page. An auditor supplies the orgId the
 * receipt belongs to, drops the JSON, and the page verifies it locally against the
 * org's published public key.
 */
export default function VerifyPage() {
  const [orgId, setOrgId] = useState('');
  const [result, setResult] = useState<VerifyBundleResult | null>(null);
  const [busy, setBusy] = useState(false);
  const [fileName, setFileName] = useState<string | null>(null);

  const onReceipt = async (parsed: unknown, name: string) => {
    setFileName(name);
    setBusy(true);
    setResult(null);
    try {
      const keyId = (parsed as ConsentReceipt)?.signature?.keyId ?? null;
      const publicKey = keyId ? await verifyApi.publicKey(keyId) : null;
      setResult(await verifyReceiptBundle(orgId.trim(), parsed, publicKey));
    } finally {
      setBusy(false);
    }
  };

  return (
    <main className={styles.page}>
      <header className={styles.header}>
        <h1>Verify a consent receipt</h1>
        <p className={styles.sub}>Independently confirm a RemoteDesk session receipt. Runs entirely in your browser.</p>
      </header>
    </main>
  );
}

```

```

<label className={styles.field}>
  <span>Organization ID (from the receipt holder)</span>
  <input value={orgId} onChange={(e) => setOrgId(e.target.value)} placeholder="org_..." />
</label>

<ReceiptDropzone onReceipt={onReceipt} />

{busy && <p className={styles.muted}>Verifying...</p>}

{result && (
  <section className={styles.result} data-ok={result.ok}>
    <h2>{result.ok ? '✓ Authentic' : '☒ Not verified'}</h2>
    <p>{REASON_TEXT[result.reason] ?? result.reason}</p>
    {result.firstBrokenSeq !== null && <p className={styles.muted}>First broken timeline entry: #
{result.firstBrokenSeq}</p>}
    <dl className={styles.facts}>
      <dt>Receipt</dt><dd className={styles.mono}>{result.facts.receiptId ?? '—'}</dd>
      <dt>Device</dt><dd className={styles.mono}>{result.facts.deviceId ?? '—'}</dd>
      <dt>Root hash</dt><dd className={styles.mono}>{result.facts.rootHash?.slice(0, 24) ?? '—'}...</dd>
      <dt>Issued</dt><dd>{result.facts.issuedAt ? new Date(result.facts.issuedAt).toLocaleString() : '—'}</dd>
      <dt>Signing key</dt><dd className={styles.mono}>{result.facts.keyId ?? '—'}</dd>
      <dt>Chain</dt><dd>{result.chainValid ? 'intact' : 'broken'}</dd>
      <dt>Signature</dt><dd>{result.signatureValid ? 'valid' : 'not valid'}</dd>
    </dl>
    {fileName && <p className={styles.muted}>Verified file: {fileName}</p>}
  </section>
)}
</main>
);
}

```

FILE: apps/web/app/verify/verify.module.css

```

.page { padding: 32px 24px; max-width: 720px; margin: 0 auto; }
.header h1 { margin: 0 0 4px; font-size: 24px; }
.sub { color: var(--muted, #6b7280); margin: 0 0 24px; }
.field { display: flex; flex-direction: column; gap: 6px; margin-bottom: 16px; font-size: 13px; }
.field input { padding: 9px 11px; border: 1px solid var(--border, #e5e7eb); border-radius: 8px; }
.dropzone { border: 2px dashed var(--border, #d1d5db); border-radius: 14px; padding: 32px; text-align: center; color: #6b7280; }
.dropzoneActive { border-color: #4f46e5; background: #eef2ff; }
.picker { color: #4f46e5; cursor: pointer; text-decoration: underline; }
.privacy { font-size: 12px; color: #9ca3af; margin-top: 12px; }
.muted { color: #6b7280; font-size: 13px; }
.mono { font-family: ui-monospace, monospace; font-size: 12px; }
.error { background: #fef2f2; color: #991b1b; padding: 8px 12px; border-radius: 8px; margin-top: 12px; }
.result { margin-top: 24px; border: 1px solid var(--border, #e5e7eb); border-radius: 14px; padding: 20px; background: #fef2f2; }
.result[data-ok='true'] { background: #ecfdf5; }
.result h2 { margin: 0 0 8px; }
.facts { display: grid; grid-template-columns: 120px 1fr; gap: 6px 12px; margin-top: 16px; font-size: 13px; }
.facts dt { color: #6b7280; }
.facts dd { margin: 0; }

```

Part D — Tests

FILE: tests/permissions/receiptVerifyBundle.test.ts

```
import { describe, it, expect, beforeAll } from 'vitest';
import { webcrypto } from 'node:crypto';
import {
  verifyReceiptBundle,
  isReceiptShaped,
  buildChain,
  signingPreimage,
  makeAuditEvent,
  type ConsentReceipt,
  type SessionSummary,
  type SigningPublicKey,
} from '@remotedesk/shared';
import { generateEd25519, signEd25519 } from '../../apps/api/src/lib/ed25519';

beforeAll(() => {
  if (!(globalThis as any).crypto) (globalThis as any).crypto = webcrypto as any;
  if (!(globalThis as any).atob) (globalThis as any).atob = (s: string) => Buffer.from(s, 'base64').toString('binary');
});

const sha = async (i: string) => {
  const d = await webcrypto.subtle.digest('SHA-256', new TextEncoder().encode(i));
  return Array.from(new Uint8Array(d), (b) => b.toString(16).padStart(2, '0')).join('');
};

const summary: SessionSummary = {
  deviceId: 'd1', startedAt: '2026-06-15T10:00:00.000Z', endedAt: '2026-06-15T10:01:00.000Z',
  durationSeconds: 60, capabilitiesUsed: { remoteInput: false, clipboardSync: false, fileTransfer: false },
  unattended: false, counts: { inputBlocked: 0, inputRateLimited: 0, emergencyStops: 0, controlGrants: 1, controlRevokes:
0 },
  endReason: 'user_disconnect',
};

async function makeSigned(orgId: string) {
  const kp = generateEd25519();
  const { lineItems, rootHash } = await buildChain([makeAuditEvent('control.granted')], sha);
  const base: ConsentReceipt = { receiptId: 'r1', summary, lineItems, rootHash, issuedAt: '2026-06-15T10:01:01.000Z',
version: 1 };
  const sig = signEd25519(kp.privateKey, signingPreimage(orgId, base));
  const receipt: ConsentReceipt = { ...base, signature: { keyId: 'k1', signature: sig, alg: 'ed25519', signedAt: 'now' }
};
  const pub: SigningPublicKey = { keyId: 'k1', publicKey: kp.publicKey, alg: 'ed25519', createdAt: 'now', retiredAt: null
};
  return { receipt, pub };
}

describe('verifyReceiptBundle', () => {
  it('shape guard rejects non-receipts', () => {
    expect(isReceiptShaped({})).toBe(false);
    expect(isReceiptShaped({ receiptId: 'r', rootHash: 'h', issuedAt: 'i', lineItems: [], summary: {} })).toBe(true);
  });

  it('verifies an authentic signed receipt', async () => {
    const { receipt, pub } = await makeSigned('org1');
```



```

    const r = await verifyReceiptBundle('org1', receipt, pub);
    expect(r.ok).toBe(true);
    expect(r.reason).toBe('ok');
    expect(r.facts.keyId).toBe('k1');
  });

  it('rejects a malformed file', async () => {
    const r = await verifyReceiptBundle('org1', { not: 'a receipt' }, null);
    expect(r.reason).toBe('malformed_receipt');
  });

  it('detects a tampered chain', async () => {
    const { receipt, pub } = await makeSigned('org1');
    receipt.lineItems[0] = { ...receipt.lineItems[0], type: 'emergency_stop.engaged' };
    const r = await verifyReceiptBundle('org1', receipt, pub);
    expect(r.reason).toBe('chain_invalid');
    expect(r.firstBrokenSeq).toBe(0);
  });

  it('reports unsigned when there is no signature', async () => {
    const { receipt } = await makeSigned('org1');
    const r = await verifyReceiptBundle('org1', { ...receipt, signature: undefined }, null);
    expect(r.reason).toBe('unsigned');
  });

  it('reports no_public_key when the key cannot be fetched', async () => {
    const { receipt } = await makeSigned('org1');
    const r = await verifyReceiptBundle('org1', receipt, null);
    expect(r.reason).toBe('no_public_key');
  });

  it('reports key_id_mismatch when the fetched key id differs', async () => {
    const { receipt, pub } = await makeSigned('org1');
    const r = await verifyReceiptBundle('org1', receipt, { ...pub, keyId: 'k2' });
    expect(r.reason).toBe('key_id_mismatch');
  });

  it('fails for the wrong org (re-attribution)', async () => {
    const { receipt, pub } = await makeSigned('org1');
    const r = await verifyReceiptBundle('org2', receipt, pub);
    expect(r.ok).toBe(false);
    expect(r.reason).toBe('signature_invalid');
  });
});

```

FILE: tests/admin/publicVerify.routes.test.ts

```

import { describe, it, expect, vi, beforeEach } from 'vitest';

const publicKeyById = vi.fn();

vi.mock('.../apps/api/src/repositories/signingKeyRepository', () => ({
  signingKeyRepository: { publicKeyById: (...a: unknown[]) => publicKeyById(...a) },
}));

import { publicVerifyRoutes } from '.../apps/api/src/routes/publicVerify.routes';

```

```

/** Minimal fake req/res to exercise the route handlers directly. */
function invoke(layerPath: string, params: Record<string, string>) {
  const layer = (publicVerifyRoutes as any).stack.find((l: any) => l.route?.path === layerPath);
  const handler = layer.route.stack[layer.route.stack.length - 1].handle;
  const req: any = { params };
  const res: any = { statusCode: 200, body: null, status(c: number) { this.statusCode = c; return this; }, json(b:
unknown) { this.body = b; return this; } };
  return new Promise((resolve) => { handler(req, res, () => resolve(res)).then?.(() => resolve(res)); setTimeout(() =>
resolve(res), 0); });
}

describe('publicVerify routes', () => {
  beforeEach(() => publicKeyById.mockReset());

  it('returns public key material for a known keyId', async () => {
    publicKeyById.mockResolvedValue({ keyId: 'k1', publicKey: 'PUB', alg: 'ed25519', createdAt: 'now', retiredAt: null
});
    const res: any = await invoke('/signing-key/:keyId', { keyId: 'k1' });
    expect(res.body.keyId).toBe('k1');
    expect(res.body.publicKey).toBe('PUB');
    // must NOT contain private material
    expect(JSON.stringify(res.body)).not.toMatch(/private|ciphertext|wrapped/i);
  });

  it('404s an unknown keyId', async () => {
    publicKeyById.mockResolvedValue(null);
    const res: any = await invoke('/signing-key/:keyId', { keyId: 'nope' });
    expect(res.statusCode).toBe(404);
  });
});

```

Part E — Reports

FILE: [generated-pack29-public-verify-portal-merge-summary.md](#)

Pack 29 (Web) — Public Receipt Verification Portal · Merge Summary

What this slice does

Opens receipt verification to third parties (auditors, end users) with NO account:

- A shared one-call verifyReceiptBundle (chain + signature, WebCrypto) for browsers.
- A PUBLIC, unauthenticated API route exposing ONLY public key material by keyId.
- A /verify web page: drop a downloaded receipt JSON + enter the org id; the page fetches the matching public key and verifies entirely in the browser.

Files created (8)

- packages/shared/src/permissions/receiptVerifyBundle.ts
- apps/api/src/routes/publicVerify.routes.ts
- apps/web/app/verify/page.tsx
- apps/web/app/verify/ReceiptDropzone.tsx
- apps/web/app/verify/verifyApi.ts
- apps/web/app/verify/verify.module.css
- tests/permissions/receiptVerifyBundle.test.ts
- tests/admin/publicVerify.routes.test.ts

Files modified (2)

- packages/shared/src/permissions/index.ts — export receiptVerifyBundle.
- apps/api/src/server.ts — mount PUBLIC /verify routes (no auth; public-key only).

Files intentionally NOT touched

- apps/desktop/** — auditor flow is web-only.
- admin signing services — keys unchanged; we only READ public keys.
- consentReceipt / receiptSignature / webcryptoEd25519 — reused as-is.

SECURITY (read this)

- /verify routes are PUBLIC by design but expose ONLY publicKeyById (public columns).
Confirm your auth layer allow-lists /verify and that NO admin/private route is mounted under it. A test asserts the response carries no private/ciphertext/wrapped fields.
- The receipt JSON is verified client-side; it is never uploaded.
- Receipts are content-free; the page shows ids/hashes/timestamps only.

Remaining manual wiring risks

1. Mount /verify BEFORE global auth (or allow-list it). It must be reachable unauthenticated.
2. WebCrypto Ed25519 must be available in auditor browsers (fails closed otherwise).
3. Consider CORS if the verify page is hosted on a different origin than the API.
4. Optionally rate-limit /verify/signing-key to deter scraping (public keys are not secret, but politeness).

Verify

- pnpm --filter @remotedesk/shared build && pnpm -r test
- Manual: download a signed receipt, open /verify, enter the org id, drop the file -> Authentic. Edit a line in the JSON -> chain broken. Change the org id -> signature not valid.

FILE: [generated-pack29-public-verify-portal-code-review.md](#)

Pack 29 (Web) — Code Review Notes

Strengths

- True independent verification: chain + Ed25519 signature run in the auditor's browser using the SAME shared verifiers as desktop + server.
- Public route exposes public key material only (publicKeyById); a test asserts no private fields leak.
- No upload: the receipt is parsed + verified locally; nothing is sent to the server except the public-key GET.
- Precise, content-free reasons (malformed / chain_invalid / unsigned / no_public_key / key_id_mismatch / signature_invalid).
- Rotation-safe: fetches the exact key by the receipt's keyId, so old receipts verify under retired keys.

Reviewer checklist

- [] /verify mounted unauthenticated (or allow-listed) and contains ONLY the public key route.
- [] No admin/private route shares the /verify prefix.
- [] CORS configured if the page is a separate origin.
- [] WebCrypto Ed25519 present in target browsers; otherwise document a polyfill.
- [] Optional rate-limit on the key route.

Safety

- Public keys only; no private/encrypted material reachable via /verify.
- Receipts remain content-free; verification reveals no session content.
- Fail-closed on every mismatch.

FILE: generated-pack29-public-verify-portal-manifest.json

```
{
  "pack": "pack29-public-verify-portal",
  "title": "Public Receipt Verification Portal for Third-Party Auditors",
```

```

"actualFileCount": 10,
"filesCreated": [
  "packages/shared/src/permissions/receiptVerifyBundle.ts",
  "apps/api/src/routes/publicVerify.routes.ts",
  "apps/web/app/verify/page.tsx",
  "apps/web/app/verify/ReceiptDropzone.tsx",
  "apps/web/app/verify/verifyApi.ts",
  "apps/web/app/verify/verify.module.css",
  "tests/permissions/receiptVerifyBundle.test.ts",
  "tests/admin/publicVerify.routes.test.ts"
],
"filesModified": [
  "packages/shared/src/permissions/index.ts",
  "apps/api/src/server.ts"
],
"filesNotTouched": [
  "apps/desktop/**",
  "apps/api/src/services/admin/**",
  "packages/shared/src/permissions/consentReceipt.ts",
  "packages/shared/src/permissions/receiptSignature.ts",
  "packages/shared/src/permissions/webcryptoEd25519.ts"
],
"dependenciesRequired": {
  "new": [],
  "existing": ["@remotedesk/shared", "express", "next", "react", "vitest", "WebCrypto (crypto.subtle Ed25519)"],
  "assumedFromProject": [
    "Pack 22 consent receipt + verifyReceipt",
    "Pack 26 receiptSignature + signingKeyRepository.publicKeyById",
    "Pack 27 rotatable keys (publicKeyById resolves retired keys)",
    "Pack 28 webcryptoEd25519 verifier"
  ]
},
"knownLimitations": [
  "Auditor must know the org id the receipt belongs to (it is not embedded to avoid implying trust); could be added as a display-only field later.",
  "WebCrypto Ed25519 required in the auditor browser; fails closed otherwise.",
  "Public key route is unauthenticated; rate-limit to deter scraping (keys are non-secret).",
  "No server-side record of third-party verifications (by design — it's client-side)."
],
"remainingManualWiringRisks": [
  "Mount /verify unauthenticated (or allow-list) with ONLY the public-key route.",
  "Configure CORS if hosting the verify page on another origin.",
  "Confirm WebCrypto Ed25519 support / add a polyfill."
],
"estimatedCompletionAfterCarefulMerge": "88-92%"
}

```

Pack 29 done: 10 files (8 created, 2 modified). Anyone can now verify a RemoteDesk consent receipt without an account. A public `/verify` page takes a downloaded receipt JSON + the org id, fetches the matching published public key, and verifies both the hash chain and the Ed25519 signature **entirely in the auditor's browser**, the file is never uploaded.

The clean bit: the whole portal is built from reuse, the browser bundle calls the exact same `verifyReceipt` (Pack 22) + `verifyReceiptSignature` (Pack 26) + `webcryptoEd25519Verify` (Pack 28) the desktop and server use, so a third party gets bit-for-bit the same verdict as the host. The public API route exposes public key material only (selected via `publicKeyById`, public columns), and a test asserts the response carries no `private` / `ciphertext` / `wrapped` fields. It's rotation-safe by fetching the exact key by the receipt's own `keyId`, so receipts signed under a retired key still verify, and every failure mode is a precise, content-free reason.

The consent-receipt system is now end-to-end provable: built (22) → verified (24) → attributed (25) → signed (26) → key-protected/rotatable (27) → host-verified + pinned (28) → publicly auditable (29).

One deploy-time security note worth repeating: mount `/verify` so it's reachable unauthenticated but make sure no admin/private route shares that prefix. Next slice = scheduled/auto key rotation with overlap windows, or move to a fresh feature area (the trust/consent/receipt system is now complete). Bolo.